Code-as-Monitor: Constraint-aware Visual Programming for Reactive and Proactive Robotic Failure Detection

Enshen Zhou[1,3*], Qi Su[2,3,4*], Cheng Chi[3*†],
Zhizheng Zhang[4], Zhongyuan Wang[3], Tiejun Huang[2,3], Lu Sheng[1†], He Wang[2,3,4†]
[1]Beihang University [2]Peking University [3]Beijing Academy of Artificial Intelligence [4]Galbot
zhouenshen@buaa.edu.cn chicheng15@mails.ucas.ac.cn lsheng@buaa.edu.cn hewang@pku.edu.cn

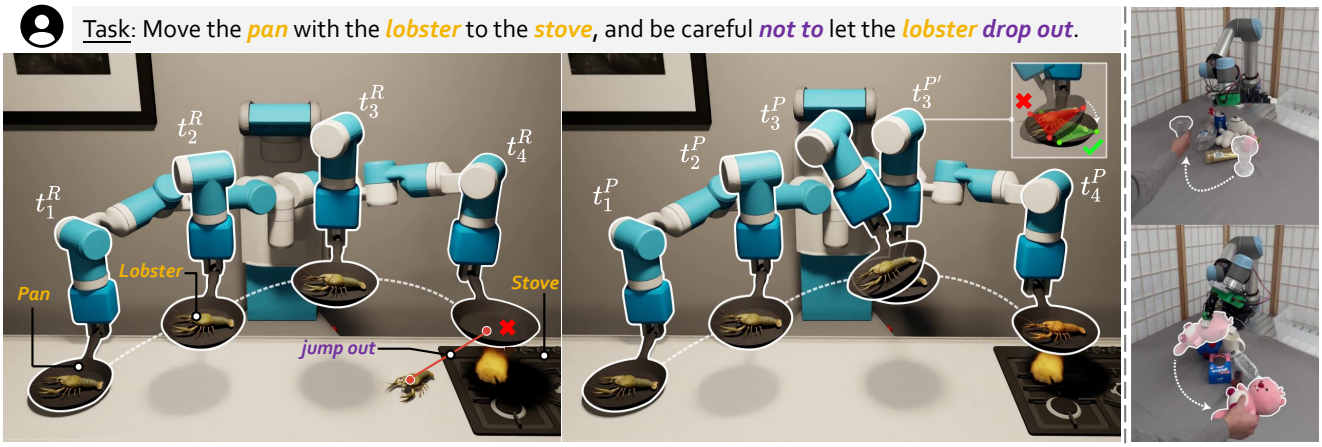

Figure 1. For the task "*Move the pan with lobster to the stove without losing the lobster*", **(a)** reactive failure detection identifies failures after they occur, and **(b)** proactive failure detection prevents foreseeable failures. In **(a)**, at t_4^R, the robot detects the failure after the lobster unpredictably jumps out due to the heat. In **(b)**, pan tilting is detected at t_3^P and corrected it at $t_3^{P'}$, requiring real-time precision. We formulate both tasks as spatio-temporal constraint satisfaction problems, leveraging our proposed constraint elements for precise, real-time checking. For example, in **(a)**, a large relative distance between pan and lobster indicates failure; in **(b)**, a large angle between the pan and the horizontal plane needs correction. **(c)** shows that our method combined with an open-loop policy forms a closed-loop system, enabling proactive (*e.g.*, detecting moving glass during grasping) and reactive (*e.g.*, removing toy after grasping) failure detection in cluttered scenes.

Abstract

Automatic detection and prevention of open-set failures are crucial in closed-loop robotic systems. Recent studies often struggle to simultaneously identify unexpected failures reactively after they occur and prevent foreseeable ones proactively. To this end, we propose Code-as-Monitor (CaM), a novel paradigm leveraging the vision-language model (VLM) for both open-set reactive and proactive failure detection. The core of our method is to formulate both tasks as a unified set of spatio-temporal constraint satisfaction problems and use VLM-generated code to evaluate them for real-time monitoring. To enhance the accuracy and efficiency of monitoring, we further introduce constraint elements that abstract constraint-related entities or their parts into compact geometric elements. This approach offers greater generality, simplifies tracking, and facilitates constraint-aware visual programming by leveraging these elements as visual prompts. Experiments show that CaM achieves a 28.7% higher success rate and reduces execution time by 31.8% under severe disturbances compared to baselines across three simulators and a real-world setting. Moreover, CaM can be integrated with open-loop control policies to form closed-loop systems, enabling long-horizon tasks in cluttered scenes with dynamic environments. See the project page at https://zhoues.github.io/Code-as-Monitor.

1. Introduction

As expectations grow for robots to handle long-horizon tasks within intricate environments, failures are unavoidable. Therefore, automatically detecting and preventing those failures plays a vital role in ensuring the tasks can

* Equal contribution † Corresponding author

eventually be solved, especially for closed-loop robotic systems. There are two modes of failure detection [30], *reactive* and *proactive*. As depicted in Fig. 1, *reactive* failure detection aims to identify failures after they occur (*e.g.*, recognizing that lobster lands on the table, *indicating* a delivery failure). In contrast, *proactive* failure detection aims to prevent foreseeable failures (*e.g.*, recognizing that a tilted pan could cause the lobster to fall out, *leading to* a delivery failure). Both detection modes are even more challenging in open-set scenarios, where the failures are not predefined.

With the help of large language models (LLMs) [14, 58] and vision-language models (VLMs) [1, 36], recent studies can achieve open-set *reactive* failure detection [10, 12, 13, 15, 20, 37, 55, 56, 66, 74] as a special case of visual question answering (VQA) tasks. However, these methods often bear compromised execution speeds and coarse-grained detection accuracy, due to the high computational costs and inadequate 3D spatio-temporal perception capability of recent LLMs/VLMs. Moreover, open-set *proactive* failure detection, which has been less explored in the literature, presents even severer challenges as it is required to foresee potential causes of failure and monitor them in real-time with high precision to anticipate and prevent imminent failure. Simply adapting LLMs/VLMs cannot meet such expectations.

In this work, we aim to develop an open-set failure detection framework that achieves reactive and proactive detection simultaneously, not only benefiting from the generalization power of VLMs but also enjoying high precision in monitoring failure characteristics with real-time efficiency. We address this by formulating both tasks as a unified set of spatio-temporal constraint satisfaction problems, which can be *precisely* translated by VLMs into executable programs. Such visual programs can efficiently verify whether entities (*e.g.*, robots, objects) or their parts in the captured environment maintain or achieve required states during or after execution (*i.e.*, satisfying constraints), so as to *immediately* prevent or detect failures. To the best of our knowledge, this is the first attempt to integrate both detection modes within a single framework. We name this constraint-aware visual programming framework as Code-as-Monitor (*CaM*).

To be specific, the proposed spatio-temporal constraint satisfaction scheme abstracts the constraint-related entity or part segments from the observed images into compact geometric elements (*e.g.*, points, lines, and surfaces), as shown in Fig. 1. It simplifies the monitoring of constraint satisfaction by tracking and evaluating the spatio-temporal combinational dynamics of these elements, eliminating the most irrelevant geometric and visual details of the raw entities/parts. The constraint element detection and tracking are grounded by our trained constraint-aware segmentation and off-the-shelf tracking models, ensuring speed, accuracy, and certain open-set adaptation capabilities. The evaluation protocol is in the form of VLM-generated code, *i.e.*, monitor code, which is visually prompted by the starting frames of a sub-goal and its associated constraint elements, in addition to the textual constraints that must be fulfilled. Once generated, this monitor code could detect reactive or proactive failures just by being executed according to the tracked constraint elements, without needing to call the VLMs again. Therefore, this minimalist scheme is generalizable to open-set failure detection for unseen entities and scenes (enabled by the potential diversity of the structured associations of constraint elements) as well as common skills (powered by the rich prior knowledge offered by VLMs), maintaining sufficient detection accuracy and real-time execution speed.

We conduct extensive experiments in three simulators (*i.e.*, CLIPort [54] , Omnigibson [31], and RLBench [22]) and one real-world setting, spanning diverse manipulation tasks (*e.g.*, pick & place, articulated objects, tool-use), robot platforms (*e.g.*, UR5, Fetch, Franka) and end-effectors (*e.g.*, suction cup, gripper, dexterous hand). The results show that *CaM* is generalizable and achieves both *reactive* and *proactive* failure detection in real-time, resulting in 28.7% higher success rates and reduced execution time by 31.8% under severe disturbances compared to the baselines. Moreover, in Sec. 4.3, *CaM* can be integrated with the existing open-loop control policy to form a closed-loop system, enabling long-horizon tasks in cluttered scenes with environment dynamics and human disturbances. Our contributions are summarized as follows:

- We introduce Code-as-Monitor (*CaM*), a novel paradigm that leverages VLMs for both *reactive* and *proactive* failure detection via constraint-aware visual programming.
- We propose the constraint elements to enhance the accuracy and efficiency of constraint satisfaction monitoring.
- Extensive experiments show that *CaM* is generalizable and achieves more real-time and precise failure detection than baselines across simulators and real-world settings.

2. Related work

Robotic Failure Detection. Recent advances in LLMs [4, 14, 58, 59] and VLMs [1, 3, 18, 33, 36, 44, 45, 52, 71, 76] greatly improve open-set *reactive* failure detection for robotics [7, 9, 23, 24, 43, 46, 57, 72] Current LLM-based methods either convert visual inputs into text [38, 53, 64], potentially losing visual details, or rely on ground-truth feedback [20, 47, 55, 56], which is impractical in real-world scenarios. Recent studies employ VLMs as failure detectors, offering binary success indicators [12, 37, 66, 74] or textual explanations [10, 13] through visual question answering (VQA), such as DoReMi [15]. However, they often bear compromised execution speeds and coarse-grained detection accuracy. Meanwhile, open-set *proactive* failure detection is rarely explored, as previous methods [2, 11] are confined to predefined failures. This detection mode requires foreseeing potential failure causes and monitoring

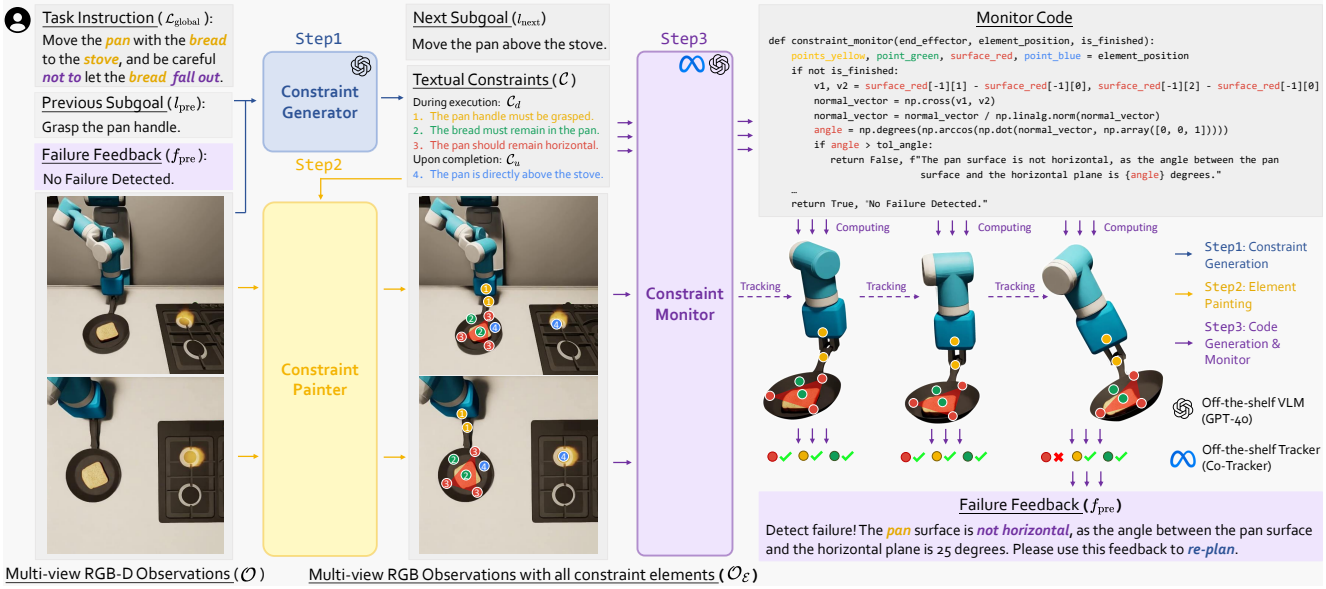

Figure 2. Overview of Code-as-Monitor. Given task instructions and prior information, the Constraint Generator derives the next subgoal and corresponding textual constraints based on multi-view observations. The Painter maps these constraints onto images as constraint elements. The Monitor generates monitor code from these images and tracks them for real-time monitoring. If any constraint is violated, it outputs the reason for failure and triggers re-planning. This framework unifies *reactive* and *proactive* failure detection via constraints, more generally abstracts relevant entities/parts through constraint elements, and ensures precise and real-time monitoring via code evaluation.

them in real-time with high precision. Herein we formulate both *reactive* and *proactive* failure detection as constraint satisfaction problems and use VLM-generated code to evaluate them, meeting the expectations of both modes above.

Visual Prompting. Visual prompts enhance the visual reasoning abilities of VLMs and are categorized into mask-based, point-based, and element-based methods. Mask-based approaches like SoM [63] apply numbered segmentation masks on images without considering instructions, whereas instruction-guided methods like IVM [73] and API [65] generate masks to block irrelevant regions. Point-based prompting encodes functionalities via points, including affordances [29, 35, 67], motion [35, 41], and constraints like ReKep [21]. However, ReKep [21] extracts semantic keypoints using DINOv2 [42] and struggles to capture precise keypoints that accurately represent desired constraints. Element-based methods like CoPa [19], represent robotic control using basic elements (*e.g.*, points, lines) via pre-trained models like SAM [27] and GPT-4V [1] simply. In contrast, this work explores constraint satisfaction monitoring and introduces constraint elements that more precisely represent relevant entities/parts. These elements are tracked and evaluated in real-time to simplify monitoring.

Visual Programming. Visual programming requires strong visual concept understanding and reasoning. Prior works show strong generalization (*e.g.*, zero-shot) across various tasks, such as image editing [8, 16], 3D visual grounding [68], and robotics control [34, 60], by integrating LLMs with visual modules but lose fine-grained details. Recent studies [40, 61] explore visual programming using VLMs, but these methods take raw RGB images as inputs with predefined primitive libraries. In contrast, this work leverages constraint elements for visual programming and uses arithmetic operations in the code to encode their spatio-temporal combinational dynamics, presenting greater challenges.

3. Method

We first give an overview of the proposed Code-as-Monitor (*CaM*) (Sec. 3.1). Then, we elaborate on the constraint element in Constraint Painter, especially constraint-aware segmentation (Sec. 3.2). Finally, we present Constraint Monitor for real-time detection (Sec. 3.3).

3.1. Overview

The proposed *CaM* comprises three key modules: the Constraint Generator, Painter, and Monitor. We focus on long-horizon manipulation task instructions $\mathcal{L}_{\text{global}}$ (*e.g.*, *"Move the pan with the bread to the stove, and be careful not to let the bread fall out"*), using RGB-D observations $\mathcal{O}$ from two camera views (front and top). As shown in Fig. 2, the RGB images $\mathcal{O}$, along with instructions $\mathcal{L}_{\text{global}}$, previous subgoal l_{pre}, and failure feedback from the Constraint Monitor f_{pre} (*e.g.*, subgoal success or failure reason), are fed into the Constraint Generator $\mathcal{F}_{\text{VLM}}$ (*i.e.*, GPT-4o [1]) to generate the next subgoal l_{next} and associated textual constraints $\mathcal{C}$. This process can be formulated as follows:

$$l_{\text{next}}, \mathcal{C}_d, \mathcal{C}_u = \mathcal{F}_{\text{VLM}}(\mathcal{O}, \mathcal{L}_{\text{global}}, l_{\text{pre}}, f_{\text{pre}}) \quad (1)$$

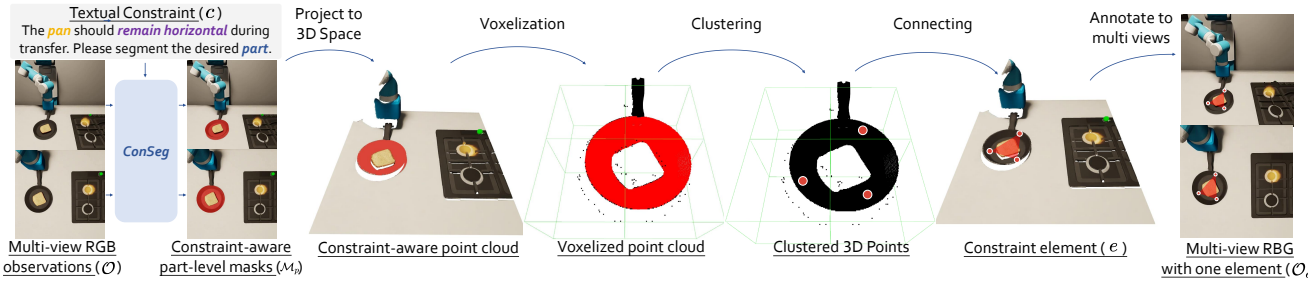

Figure 3. Constraint Element Pipeline. Given a constraint, our model *ConSeg* generates instance-level and part-level masks across multiple views, which are projected into 3D space. Through a series of heuristics, the desired elements are produced. Once all elements are obtained, they are annotated onto the original multi-view images. Here we display the annotation result of one element.

where $\mathcal{C}_d = \{c_d^0, c_d^1, \ldots, c_d^n\}$ denotes the constraints that must be maintained during subgoal execution (*e.g.*, pan handle must be grasped, bread must remain in the pan, pan should remain horizontal during transfer), and $\mathcal{C}_u = \{c_u^0, c_u^1, \ldots, c_u^k\}$ refers to the constraints that must be met upon subgoal completion (*e.g.*, pan should be directly above the stove). We successfully unify *reactive* and *proactive* failure detection as these task-specific, situation-aware constraint satisfaction problems.

In Painter, for each textual constraint c from $\mathcal{C}_d$ or $\mathcal{C}_u$, we generate corresponding constraint elements e (detailed in Sec. 3.2) from observations $\mathcal{O}$. These elements, which are composed of 3D points, abstract the constraint-related entities or their parts to represent the desired textual constraint more easily (*e.g.*, the constant distance between green points on bread and pan determines if bread remains in pan, as shown in Fig. 2.) These generated elements are then aggregated into the final set $\mathcal{E} = \{e^0, e^1, \ldots, e^{n+k}\}$, and numerically annotated across all views to produce the final visual prompted images $\mathcal{O}_\mathcal{E}$.

In Monitor, we provide GPT-4o [1] with the next subgoal l_{next}, textual constraints $\mathcal{C}$, and annotated observations $\mathcal{O}_\mathcal{E}$ for constraint-aware visual programming to generate the evaluation protocol, *i.e.*, monitor code. This code inputs the elements' 3D positions, calculates arithmetic operations within it, and returns a boolean to indicate potential or actual failure and a string to describe its reason. During subgoal execution, Monitor tracks the elements and evaluates the spatio-temporal combinational dynamics of these elements. If the code returns `False`, the policy execution halts immediately, and the accompanying string is used as feedback f_{pre} for re-planning. Otherwise, the subgoal is considered completed. In either case, the cycle is repeated.

3.2. Constraint Element

To simplify the monitoring of constraint satisfaction, we introduce constraint elements by abstracting constraint-related entities/parts into compact geometric elements (*e.g.*, points, lines) to get rid of the most irrelevant geometric and visual details, making them easier to track and generalize.

Pipeline. The constraint element generation pipeline is shown in Fig. 3. Our trained multi-granularity constraint-aware model *ConSeg* performs two inference steps for each c and each RGB image o from the set of views O. First, instance-level masks $\mathcal{M}_i$ are generated to capture constraint-related entities. Then, fine-grained part-level masks $\mathcal{M}_p$ and corresponding element type descriptions l_{e} are produced, as shown in Fig. 4. Using corresponding depth data, we project $\mathcal{M}_p$ from all views into 3D space, fusing them into a point cloud. However, directly tracking and evaluating the spatio-temporal combinational dynamics of these constraint-related entities/parts is challenging. Therefore, we convert these entities into proposed constraint elements. We first apply voxelization to the point cloud with voxel size determined by element type l_{e} (*e.g.*, the surface needs at least 3 points, we divide the occupied space into 2×2 voxels.). Then we cluster one representative point within each voxel and filter them to a specified number, also determined by l_{e} to extract the final desired 3D points. We connect points within each instance-level mask $\mathcal{M}_i$ to form the constraint element e associated with constraint c. Notably, points modeled as end-effectors (*e.g.*, the points of the fingertips and hand's center represent a dexterous hand) can be directly obtained from forward kinematics, bypassing the process above. Additionally, we perform parallel inference of *ConSeg* across all views to expedite the acquisition of the final set $\mathcal{E}$ and their annotations onto the corresponding views $\mathcal{O}_\mathcal{E}$. Moreover, this minimalist approach, *i.e.*, constraint elements, emphasizes the most relevant entities/parts, enabling generalization to unseen scenes and entities, which is critical for open-set failure detection. For more details, please check Supp. B.3.

Constraint-aware Segmentation. Since the textual constraint does not explicitly specify relevant entities/parts, we require a model capable of logical reasoning and certain open-set adaptation, enabling precise constraint-aware instance and part-level segmentation. As depicted in Fig. 4, we propose *ConSeg*, which builds upon LISA [28] comprising a VLM $\mathcal{F}_{\text{VLM}}$ (*i.e.*, LLaVA [36]) with the vision encoder $\mathcal{F}_{\text{enc}}$ and decoder $\mathcal{F}_{\text{dec}}$ from SAM [27]. The textual

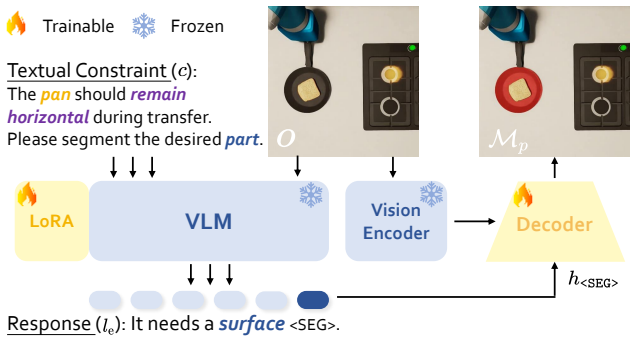

Figure 4. *ConSeg* architecture. Here we display the part-level segmentation, which will output the desired element type and mask.

constraint c and the image o are input into the VLM $\mathcal{F}_{\text{VLM}}$ to generate an embedding for the <SEG> token. The VLM outputs a textual description l_{e} of the desired element type additionally for part-level segmentation. The decoder $\mathcal{F}_{\text{dec}}$ then generates the segmentation mask $\mathcal{M}$ by leveraging visual features from the vision encoder $\mathcal{F}_{\text{enc}}$ based on the current observation o, and features derived from the final-layer embedding $h_{<\text{SEG}>}$ of the <SEG> token, transformed via an MLP ($\mathcal{F}_{\text{MLP}}$). The process can be formulated as follows:

$$l_{\text{e}}, h_{<\text{SEG}>} = \mathcal{F}_{\text{VLM}}(c, o), \mathcal{M} = \mathcal{F}_{\text{dec}}(\mathcal{F}_{\text{MLP}}(h_{<\text{SEG}>}), \mathcal{F}_{\text{enc}}(o)) \quad (2)$$

The entire pipeline, including additional LoRA [17] parameters, is optimized end-to-end while keeping the parameters of the VLM $\mathcal{F}_{\text{VLM}}$ and vision encoder $\mathcal{F}_{\text{enc}}$ frozen.

Dataset and Collection Pipeline. Our dataset utilizes images from BridgeData V2 [62]. While BridgeData V2 provides trajectory-level instructions, we require frame-level annotations for per-frame subgoals and constraints. To address this, we sample pick-and-place data, using external references (*e.g.*, gripper open/close states) to generate $10,181$ trajectories with $219,356$ images. We employ GPT-4o to decompose trajectory instructions into subgoals, constraints, and object-part associations to generate ground-truth annotations. Instance-level and part-level segmentations are performed using Grounded SAM [49] and Semantic SAM [32], respectively. These outputs are integrated and refined through manual inspection to produce the multi-granularity dataset, which is combined with LISA's training data to fine-tune our model. More details are in Supp. B.

3.3. Real-time Monitor Module

After annotating constraint elements with unique colors and numerical labels across all views, these images serve as visual prompts to generate monitor code via GPT-4o [1] with constraints $\mathcal{C}$ and the next subgoal l_{next}. The generated code accepts the elements' 3D positions as input and performs arithmetic operations (*e.g.*, NumPy-supported calculations) to evaluate spatio-temporal constraints at various stages (*i.e.*, during execution or upon completion), as shown in Fig. 2 code block. It returns a boolean flag indicating potential or actual failure if the computed result exceeds a predefined tolerance (*e.g.*, accepting deviations of the pan's surface normal from the z-axis within $15°$), along with a string explaining the cause. By tracking the elements using CoTracker [25], we achieve real-time detection via VLMs without frequent calls, reducing the computational cost.

Notably, including both current positions and historical trajectories of the elements as input allows us to monitor tasks needing high precision, such as the requirement of 2cm movement or $180°$ rotation, which previous *reactive* detection methods could not address through direct VLM queries using raw RGB images. Moreover, the potential diversity in the structured associations of constraint elements within the code (*e.g.*, calculating the angle between the normal vector of the pan's surface element and the z-axis determines if the pan is level, as shown in Fig. 2 code block) and rich prior knowledge offered by VLMs enable generalization to certain unseen tasks requiring common skills.

4. Experiments

Our experiments aim to address the following questions: **(1)** Can *CaM* achieve open-set *reactive* and *proactive* failure detection across diverse robots, end-effectors, and objects, both in simulator (Sec. 4.2) and on real robots (Sec. 4.3)? **(2)** Can *ConSeg* infer multi-granularity constraint-aware masks for unseen scenes and objects (Sec. 4.4)? **(3)** Which design choices greatly enhance the performance (Sec. 4.5)?

4.1. Experimental Setup

Environment Settings. We evaluate *CaM* across three simulators (*i.e.*, CLIPort [54], Omnigibson [31], and RLBench [22]) and a real-world setting. In CLIPort, we employ a UR5 arm equipped with a suction cup for pick-and-place and a spatula for pushing, controlled by pre-trained CLIPort policy [54]. Omnigibson features a Fetch robot with a gripper, controlled by ReKep [21]. RLBench uses a Franka arm with a gripper, controlled by ARP [70]. We collect a small dataset to fine-tune *ConSeg* for each simulator. We use a UR5 arm with a Leap Hand [51] for real-world experiments, controlled by an open-loop policy named DexGraspNet 2.0 [69]. More details are provided in Supp. C.

Task Settings. We introduce disturbances across all environments to induce failures, classified by the affected constraints (*e.g.*, points, lines, surfaces). We evaluate task success rate, execution time, and additional token usage in Omnigibson. Notably, we don't assess failure judgment success rate because our code operates continuously in real-time, rendering this metric inapplicable. Detailed evaluation settings are provided in each section.

Baselines. We compare DoReMi (DRM) [15] as the baseline for all settings, which uses VLMs to detect intermediate failures during execution via frequent VQA-based queries.

In CLIPort, we include Inner Monologue [20] baseline, which detects failures only upon each subgoal completion.

4.2. Main Results in Simulator

4.2.1 Results in CLIPort

We evaluate two tasks in CLIPort: **(1)** Stack in Order: The robot must stack blocks in a specified order, including two point-level disturbances: (a) with per-step probability p, the suction cup may release a block, causing it to drop; (b) placement positions are perturbed by uniform noise in $[0, q]$ cm, potentially leading to tower collapse. Success is defined as correctly stacking the blocks within 70s. **(2)** Sweep Half the Blocks: To address the pre-trained policy's tendency to sweep all blocks, the robot must determine when to halt, sweeping half of the blocks ($\pm 10\%$) into a specified colored area within 30s. Success is achieved by meeting these criteria. Tab. 1 shows the mean results over 5 different seeds, each with 12 episodes. Check Supp. D.1 for details.

Code better monitors 3D space relations. As shown in the Tab. 1, Under the most severe disturbances (p=0.3, q=3) in "Stack in Order", *CaM* achieves a 17.5% higher success rate than DoReMi. Frequent VLM queries lead to increased incorrect failure judgments due to a limited 3D spatial understanding from single images, compared to code-based evaluation of block positions. For example, after placing the green block on the red and preparing to pick up the blue, DoReMi mistakenly thinks that the green block is no longer atop the red, causing it to re-execute the previous subgoal.

Code with elements achieve *reactive* and *proactive* failure detetction. As shown in Tab. 1, under severe disturbances in "Stack in Order", *CaM* reduces execution time by 38.7% and 14.4% compared to Inner Monologue [20], which only detects failures upon subgoal completion, and DoReMi [15], which suffers from misjudgments due to repeated VLM queries, respectively. In contrast, *CaM* unifies both *reactive* and *proactive* failure detection with high precision in real-time, especially in preventing foreseeable failures. For example, if the green block is placed on the red block with heavy placement position disturbance, *CaM* anticipates that further stacking the blue may lead to collapse and then stabilizes the green block before proceeding.

Code with elements leads to more accurate counting. As shown in the Tab. 1, in "Sweep Half the Blocks", *CaM* achieves average success rates $4.5\times$ higher than DoReMi [15]. Calculating the block points in the designated surface region provides more accurate results than directly using the VLM to count, enabling a more precise halting of the policy to complete the task.

4.2.2 Results in Omnigibson

We conduct experiments on three tasks in Omnigibson, each involving a distinct type of constraint-element disturbance:

Table 1. Performance in CLIPort. We report the success rate and execution time, compared to CLIPort (CP) [54], with Inner Monologue (IM) [20] and DoReMi (DRM) [15].

Tasks with disturbance		Success Rate(%) ↑				Execution Time(s) ↓		
		CP	+IM	+DRM	**+Ours**	+IM	+DRM	**+Ours**
Stack in	p=0.0	100.0	100.0	100.0	100.0	13.4	13.4	13.4
order with	p=0.15	56.67	81.67	83.33	**95.00**	34.8	26.00	**21.0**
drop p	p=0.3	21.67	75.00	76.67	**88.33**	42.8	34.20	**25.4**
Stack in	q=1	90.00	90.00	96.67	**98.33**	24.8	24.6	**24.2**
order with	q=2	41.67	71.67	75.00	**83.33**	39.4	37.0	**29.2**
noise q	q=3	15.00	40.00	40.00	**63.33**	58.2	54.2	**36.8**
Sweep Half the Blocks		0.00	18.33	16.67	**75.00**	22.0	16.6	**16.4**

(1) Slot Pen: Insert a pen into a holder, facing point-level disturbances wherein (a) pen is moved during grasping; (b) pen is dropped during transport; (c) holder is moved during insertion. **(2)** Stow Book: Place a book to a bookshelf vertically, with line-level disturbances where (a) book is randomly rotated during grasping; (b) end-effector joint is randomly actuated to alter the book pose; (c) book is reoriented horizontally after placement. **(3)** Pour Tea: Pours from a teapot into a teacup, encountering surface-level disturbances wherein (a) teapot is tilted forward/backward during movement; (b) end-effector joint is actuated to induce a lateral tilt of the teapot during movement; (c) teapot is returned to a horizontal position during pouring. Tab. 2 reports the results across three tasks, each including one no-disturbance trial and three specific-disturbance trials, with 10 runs for each setting. More details are provided in Supp. D.2.

Code with elements detects richer failures. In Tab. 2, only *CaM* can detect failures caused by surface-level disturbances in "Pour Tea" compared to DoReMi [15]. The reason is that changes in the teapot's pitch and roll angles are hard to detect by querying VLM via VQA using one current image. The misjudgment leads to a 0% success rate of DoReMi in this task. Notably, DoReMi's success rate is sometimes lower than that of ReKep alone due to VLM's limited spatio-temporal understanding of single images.

Code with elements detects failure with lower computational cost. As shown in Tab. 2, we achieve a 34.8% reduction in execution time and a 52.2% decrease in token count compared to DoReMi [15]. This improvement stems from *proactive* failure detection, which prevents more severe failures ahead in real-time for timely re-planning while generating code only once per subgoal. For example, in "Pour Tea", *CaM* detects failure by monitoring the teapot's tilt angle in real-time, avoiding frequent VLM checks.

4.2.3 Results in RLBench

We further evaluate *CaM* on RLbench and demonstrate its superior generalization. Details can be found in Supp. D.3.

4.3. Main Results in Real World

We conduct real-world evaluations on two tasks: **(1)** Simple Pick & Place: The robot has 70s to pick up objects and

Table 2. Performance in Omnigibson. We report the success rate (SR), execution time, and token usage, compared to DoReMi (DRM) [15].

Method	Slot Pen (Point-level Disturbances)						Stow Book (Line-level Disturbances)						Pour Tea (Surface-level Disturbances)					
	SR with Disturbance(%) ↑				Time	Token	SR with Disturbance(%) ↑				Time	Token	SR with Disturbance(%) ↑				Time	Token
	None	Dist.(a)	Dist.(b)	Dist.(c)	(s)↓	(k)↓	None	Dist.(a)	Dist.(b)	Dist.(c)	(s)↓	(k)↓	None	Dist.(a)	Dist.(b)	Dist.(c)	(s)↓	(k)↓
ReKep [21]	30	20	10	10	-	-	40	30	30	20	-	-	20	20	20	10	-	-
+DRM	40	10	20	20	177.84	54.54	50	40	20	40	127.17	38.67	0	0	0	0	-	-
+Ours	**60**	**50**	**40**	**40**	**101.85**	**25.82**	**70**	**60**	**70**	**60**	**93.08**	**18.67**	**50**	**40**	**40**	**30**	**174.55**	**44.19**

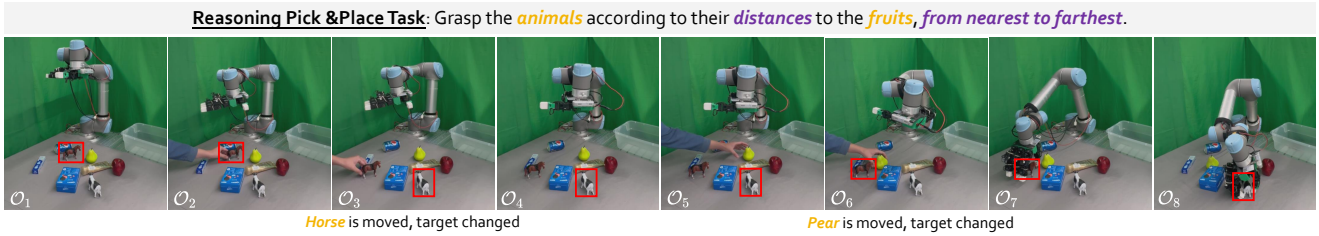

Figure 5. Example of Real-world Evaluation. The red bounding box shows the current grasp target, which may shift due to environmental changes. *CaM* monitors and adapts to these changes in real-time, resulting in a closed-loop system with an open-loop policy.

Table 3. Performance of Single Pick & Place. We report the success rate and execution time. DGN donates DexGraspNet 2.0 [69].

Tasks with disturbance	Object types	Success Rate(%) ↑			Execution Time(s) ↓	
		DGN	+DRM	**+Ours**	+DRM	**+Ours**
Pick & Place with the objects being moved	Deformable	0.00	83.33	**96.67**	61.8	**46.3**
	Transparent	0.00	66.67	**93.33**	72.6	**48.1**
	Small Rigid	0.00	80.0	**96.67**	65.7	**45.4**
	Large Geometric	0.00	86.67	**96.67**	68.9	**45.3**
Pick & Place with the objects being removed	Deformable	0.00	76.67	**93.33**	68.7	**62.5**
	Transparent	0.00	60.00	**90.00**	77.7	**62.7**
	Small Rigid	0.00	63.33	**93.33**	69.8	**60.5**
	Large Geometric	0.00	76.67	**96.67**	72.3	**60.3**

Table 4. Performance of Reasoning Pick & Place in cluttered scene. We report the success rate. The robot is controlled by an open-loop policy named DexGraspNet 2.0 (DGN) [69]. w/ CE with ✓ indicates using constraint elements; otherwise, constraint-aware entities or parts are used for tracking and code computation.

Tasks	w/ CE	Success Rate(%) ↑		
		DGN	+DRM	**+Ours**
Clear all objects on table except for animals	✓	0.00	10.00	**60.00**
	✗	0.00	10.00	**20.00**
Grasp the animals according to their distances to fruits, from nearest to farthest	✓	0.00	0.00	**90.00**

place them at specified locations, facing two disturbances: (a) moving the object during grasping, and (b) removing the object from hand during movement. We test four object types, selecting 3 examples per type and conducting 10 trials for each (see Tab. 3). **(2)** Reasoning Pick & Place: The robot executes long-horizon tasks, involving ambiguous terms (*e.g.*, "fruit"), under the same disturbances. We evaluate 2 long-horizon tasks in cluttered scenes, performing 10 trials each (see Tab. 4). Check Supp. D.4 for details.

Elements generally abstract constraints and relevant entities. As shown in Tab. 3, using a different end-effector (*i.e.*, Leap Hand [51]) in Simple Pick & Place, *CaM* also achieves success rates surpassing DoReMi [15] by 20.4% when handling different kinds of objects (*e.g.*, deformable). We find that abstracting constraint-related entities/parts removes the irrelevant visual details, enabling generalization to different kinds of entities in unseen scenes (as discussed in Sec. 3.2), leading to easier tracking and code evaluation.

Table 5. Performance of reasoning and constraint-aware segmentation. FMC denotes the foundation model combination baseline.

Method	ReasonSeg		ConstraintSeg			
	Instance-level		Instance-level		Part-level	
	gIoU	cIoU	gIoU	cIoU	gIoU	cIoU
PixelLM [50]	56.0	61.4	44.4	43.2	24.1	22.6
LISA-13B [28]	**56.6**	**65.1**	42.1	38.9	23.4	24.3
FMC	51.2	53.3	49.3	49.6	40.8	39.3
***ConSeg*-13B**	55.7	63.9	**62.1**	**68.7**	**60.2**	**65.3**

Table 6. Ablation studies in Omnigibson's "Stow Book", assessing the impact of Multi-View (MV), Constraint-aware Segmentation (CS) for elements, and Connect Points (CP) for element formation.

MV	CS	CP	Success Rate with Disturbance(%) ↑				
			None	Dist.(a)	Dist.(b)	Dist.(c)	Avg
✗	✓	✓	40.0	40.0	30.0	50.0	40.0
✓	✗	✓	50.0	40.0	40.0	40.0	42.5
✓	✓	✗	60.0	50.0	60.0	50.0	55.0
✓	✓	✓	**70.0**	**60.0**	**70.0**	**60.0**	**65.0**

***Reactive* and *Proactive* failure detection combined with an open-loop policy forms a close-loop system.** Tab. 4 shows that only *CaM* successfully handles long-horizon tasks in cluttered scenes. These tasks are challenging as the robot is controlled by an open-loop policy that cannot handle environment dynamics and human disturbances in a closed-loop manner. By incorporating both *reactive* and *proactive* failure detection with the open-loop policy (see Fig. 5), the robot can dynamically adjust its target in real-time. For example, when a human moves the horse or pear during the task, the robot adapts by grasping the animal closest to the fruit, forming a closed-loop system.

4.4. Main Results of Segmentation

Tab. 5 presents segmentation results comparing our *ConSeg*, with SOTA models and a Foundation Model Combination (FMC) baseline. FMC integrates GPT-4o for reasoning over tasks and constraints to identify relevant instances and parts, along with Grounded SAM [49] and Semantic SAM [32] for instance and part-level segmentation, respectively, similar to our data collection pipeline. We evaluate performance on the ReasonSeg [28] benchmark and our

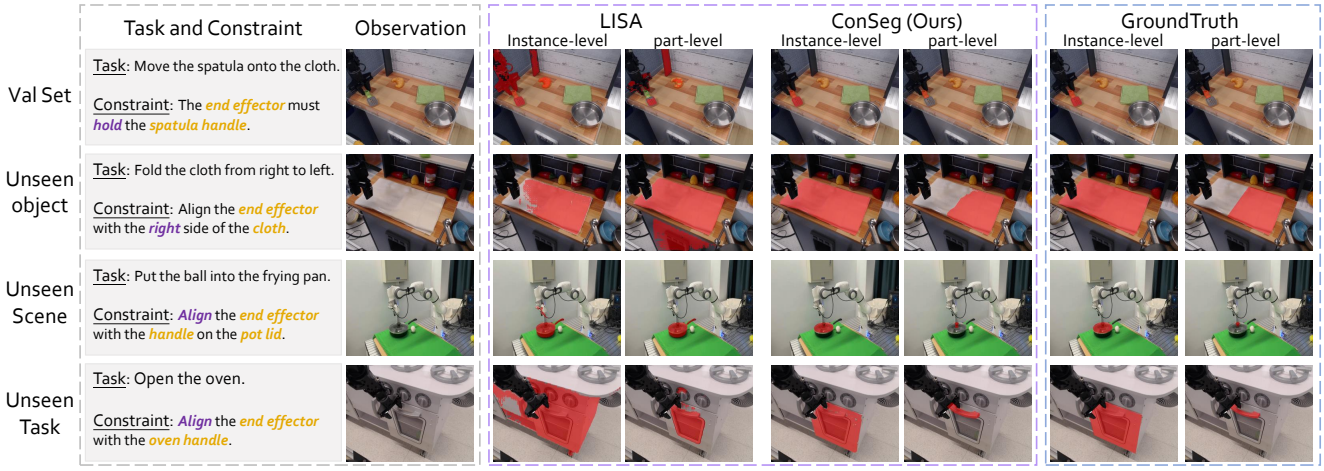

Figure 6. Visual comparison between our *ConSeg* and LISA [28] at instance and part level. The red masks are the segmentation results.

Table 7. Ablation study on training data. SemanticSeg includes ADE20K [75], COCO-Stuff [5], PACO-LVIS [48] and PASCAL-Part [6]. ReferSeg includes refCLEF, refCOCO, refCOCO+ [26] and refCOCOg [39]. VQA indicates LLaVAInstruct-150k [36].

SemanticSeg	ReferSeg	VQA	ReasonSeg	ContraintSeg		Ins-level		Part-level	
				Ins	Part	gIoU	cIoU	gIoU	cIoU
✓	✓	✓	✓	✗	✗	42.1	38.9	23.4	24.3
✓	✓	✓	✓	✓	✗	60.7	65.9	40.4	45.6
✓	✓	✓	✓	✗	✓	51.5	50.6	56.5	61.7
✓	✓	✓	✓	✓	✓	**62.1**	**68.7**	**60.2**	**65.3**

proposed Constraint-Aware Segmentation (ConstraintSeg) benchmark. Our benchmark evaluates performance using gIoU and cIoU, following ReasonSeg's evaluation setting.

***ConSeg* performs both reasoning and multi-granularity constraint-aware segmentation.** As shown in Tab. 5, *ConSeg* performs comparably to LISA and PixelLM on the ReasonSeg but significantly surpasses them on ConstraintSeg, achieving nearly a 40% improvement at the part level. Visual comparisons in Fig. 6 further demonstrate *ConSeg*'s strong generalization to unseen objects, scenes, and tasks.

4.5. Ablation Study

We conduct ablation studies and present analyses below.

Multi views are critical for visual programming. As shown in Tab. 6, using only front-view images with constraint elements reduces the average success rate from 65% to 40%. This is primarily due to (1) occlusion leading to suboptimal element generation and (2) errors in visual programming, where dimensional reduction causes surfaces to be misinterpreted as lines or lines as points.

Constraint-aware segmentation enhances element. As shown in Tab. 6, using DINOv2 [42] to generate semantic points to form elements, rather than through constraint-aware segmentation, reduces the success rate from 65% to 42.5%, because these constructed elements fail to represent the desired constraints accurately. For example, capturing a book's vertical orientation requires two precise points on its edge, which DINOv2 can not provide.

Forming elements improves code generation. As shown in Tab. 6, using unconnected 3D points as final elements reduces the success rate from 65% to 55%. Pre-formed elements contain more prior constraint information, serving as visual cues better encoded in code, whereas standalone points require recombination during visual programming.

Elements simplify constraints computation. As shown in Tab. 4, the success rate decreases from 60% to 20% when entities or parts are not transformed into proposed elements in real-world evaluation. We find two key issues: (1) inaccurate 3D position and slow pose tracking of entities or parts; and (2) difficulties in performing arithmetic operations on their 3D positions or poses in code, hindering the encoding of their spatio-temporal constraints.

Both instance-level and part-level data improve performance. We conduct ablation studies to assess the impact of our proposed dataset on the ConstraintSeg benchmark. The results are shown in Tab. 7. The results indicate that both the instance-level and part-level subsets contribute to performance improvements and enhance each other's effects.

5. Conclusion

In this paper, we present a novel paradigm termed Code-as-Monitor, leveraging the VLMs for both open-set reactive and proactive failure detection. In detail, We formulate both detection modes as spatio-temporal constraint satisfaction problems and use VLM-generated code to evaluate them for real-time monitoring. We further propose constraint elements, which abstract constraints-related entities or their parts into compact geometric elements, to improve the precision and efficiency of monitoring. Extensive experiments demonstrate the superiority of the proposed approach and highlight its potential to advance closed-loop robot systems.

Acknowledgement

This work was supported by the National Natural Science Foundation of China (62132001), the National Science and Technology Major Project (2022ZD0116314), and the Fundamental Research Funds for the Central Universities.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv*, 2023. 2, 3, 4, 5

[2] Aneseh Alvanpour, Sumit Kumar Das, Christopher Kevin Robinson, Olfa Nasraoui, and Dan Popa. Robot failure mode prediction with explainable machine learning. *CASE*, 2020. 2

[3] Jingkun An, Yinghao Zhu, Zongjian Li, Enshen Zhou, Haoran Feng, Xijie Huang, Bohua Chen, Yemin Shi, and Chengwei Pan. Agfsync: Leveraging ai-generated feedback for preference optimization in text-to-image generation. *arXiv preprint arXiv:2403.13352*, 2024. 2

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *NeurIPS*, 2020. 2

[5] Holger Caesar, Jasper Uijlings, and Vittorio Ferrari. Cocostuff: Thing and stuff classes in context. *CVPR*, 2018. 8

[6] Xianjie Chen, Roozbeh Mottaghi, Xiaobai Liu, Sanja Fidler, Raquel Urtasun, and Alan Yuille. Detect what you can: Detecting and representing objects using holistic models and body parts. *CVPR*, 2014. 8

[7] Zeren Chen, Zhelun Shi, Xiaoya Lu, Lehan He, Sucheng Qian, Zhenfei Yin, Wanli Ouyang, Jing Shao, Yu Qiao, Cewu Lu, et al. Rh20t-p: A primitive-level robotic dataset towards composable generalization agents. *arXiv preprint arXiv:2403.19622*, 2024. 2

[8] Jaemin Cho, Abhay Zala, and Mohit Bansal. Visual programming for step-by-step text-to-image generation and evaluation. *NeurIPS*, 2024. 3

[9] Wenbo Cui, Chengyang Zhao, Songlin Wei, Jiazhao Zhang, Haoran Geng, Yaran Chen, and He Wang. Gapartmanip: A large-scale part-centric dataset for material-agnostic articulated object manipulation. *arXiv preprint arXiv:2411.18276*, 2024. 2

[10] Yinpei Dai, Jayjun Lee, Nima Fazeli, and Joyce Chai. Racer: Rich language-guided failure recovery policies for imitation learning. *arXiv*, 2024. 2

[11] Maximilian Diehl and Karinne Ramirez-Amaro. A causal-based approach to explain, predict and prevent failures in robotic tasks. *RAS*, 2023. 2

[12] Yuqing Du, Ksenia Konyushkova, Misha Denil, Akhil Raju, Jessica Landon, Felix Hill, Nando de Freitas, and Serkan Cabi. Vision-language models as success detectors. *arXiv*, 2023. 2

[13] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *arXiv*, 2024. 2

[14] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv*, 2024. 2

[15] Yanjiang Guo, Yen-Jen Wang, Lihan Zha, Zheyuan Jiang, and Jianyu Chen. Doremi: Grounding language model by detecting and recovering from plan-execution misalignment. *arXiv*, 2023. 2, 5, 6, 7

[16] Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. *CVPR*, 2023. 3

[17] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 2021. 5

[18] Xuhao Hu, Dongrui Liu, Hao Li, Xuanjing Huang, and Jing Shao. Vlsbench: Unveiling visual leakage in multimodal safety. *arXiv preprint arXiv:2411.19939*, 2024. 2

[19] Haoxu Huang, Fanqi Lin, Yingdong Hu, Shengjie Wang, and Yang Gao. Copa: General robotic manipulation through spatial constraints of parts with foundation models. *arXiv*, 2024. 3

[20] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv*, 2022. 2, 6

[21] Wenlong Huang, Chen Wang, Yunzhu Li, Ruohan Zhang, and Li Fei-Fei. Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. *arXiv*, 2024. 3, 5, 7

[22] Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J. Davison. Rlbench: The robot learning benchmark & learning environment. *RAL*, 2020. 2, 5

[23] Yuheng Ji, Huajie Tan, Jiayu Shi, Xiaoshuai Hao, Yuan Zhang, Hengyuan Zhang, Pengwei Wang, Mengdi Zhao, Yao Mu, Pengju An, et al. Robobrain: A unified brain model for robotic manipulation from abstract to concrete. *arXiv preprint arXiv:2502.21257*, 2025. 2

[24] Yueru Jia, Jiaming Liu, Sixiang Chen, Chenyang Gu, Zhilue Wang, Longzan Luo, Lily Lee, Pengwei Wang, Zhongyuan Wang, Renrui Zhang, et al. Lift3d foundation policy: Lifting 2d large-scale pretrained models for robust 3d robotic manipulation. *arXiv preprint arXiv:2411.18623*, 2024. 2

[25] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. Cotracker: It is better to track together. *ECCV*, 2024. 5

[26] Sahar Kazemzadeh, Vicente Ordonez, Mark Matten, and Tamara Berg. Referitgame: Referring to objects in photographs of natural scenes. *EMNLP*, 2014. 8

[27] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. *ICCV*, 2023. 3, 4

[28] Xin Lai, Zhuotao Tian, Yukang Chen, Yanwei Li, Yuhui Yuan, Shu Liu, and Jiaya Jia. Lisa: Reasoning segmentation via large language model. *CVPR*, 2024. 4, 7, 8

[29] Olivia Y Lee, Annie Xie, Kuan Fang, Karl Pertsch, and Chelsea Finn. Affordance-guided reinforcement learning via visual prompting. *arXiv*, 2024. 3

[30] Gregory LeMasurier, Alvika Gautam, Zhao Han, Jacob W Crandall, and Holly A Yanco. Reactive or proactive? how robots should explain failures. *HRI*, 2024. 2

[31] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Michael Lingelbach, Jiankai Sun, Mona Anvari, Minjune Hwang, Manasi Sharma, Arman Aydin, Dhruva Bansal, Samuel Hunter, Kyu-Young Kim, Alan Lou, Caleb R Matthews, Ivan Villa-Renteria, Jerry Huayang Tang, Claire Tang, Fei Xia, Silvio Savarese, Hyowon Gweon, Karen Liu, Jiajun Wu, and Li Fei-Fei. Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation. *CoRL*, 2022. 2, 5

[32] Feng Li, Hao Zhang, Peize Sun, Xueyan Zou, Shilong Liu, Jianwei Yang, Chunyuan Li, Lei Zhang, and Jianfeng Gao. Semantic-sam: Segment and recognize anything at any granularity. *arXiv*, 2023. 5, 7

[33] Lijun Li, Zhelun Shi, Xuhao Hu, Bowen Dong, Yiran Qin, Xihui Liu, Lu Sheng, and Jing Shao. T2isafety: Benchmark for assessing fairness, toxicity, and privacy in image generation. *arXiv preprint arXiv:2501.12612*, 2025. 2

[34] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. *ICRA*, 2023. 3

[35] Fangchen Liu, Kuan Fang, Pieter Abbeel, and Sergey Levine. Moka: Open-vocabulary robotic manipulation through mark-based visual prompting. *arXiv*, 2024. 3

[36] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *NeurIPS*, 2024. 2, 4, 8

[37] Jiaming Liu, Chenxuan Li, Guanqun Wang, Lily Lee, Kaichen Zhou, Sixiang Chen, Chuyan Xiong, Jiaxin Ge, Renrui Zhang, and Shanghang Zhang. Self-corrected multimodal large language model for end-to-end robot manipulation. *arXiv*, 2024. 2

[38] Zeyi Liu, Arpit Bahety, and Shuran Song. Reflect: Summarizing robot experiences for failure explanation and correction. *CoRL*, 2023. 2

[39] Junhua Mao, Jonathan Huang, Alexander Toshev, Oana Camburu, Alan L Yuille, and Kevin Murphy. Generation and comprehension of unambiguous object descriptions. *CVPR*, 2016. 8

[40] Yao Mu, Junting Chen, Qinglong Zhang, Shoufa Chen, Qiaojun Yu, Chongjian Ge, Runjian Chen, Zhixuan Liang, Mengkang Hu, Chaofan Tao, et al. Robocodex: Multimodal code generation for robotic behavior synthesis. *arXiv*, 2024. 3

[41] Soroush Nasiriany, Fei Xia, Wenhao Yu, Ted Xiao, Jacky Liang, Ishita Dasgupta, Annie Xie, Danny Driess, Ayzaan Wahid, Zhuo Xu, et al. Pivot: Iterative visual prompting elicits actionable knowledge for vlms. *arXiv*, 2024. 3

[42] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv*, 2023. 3, 8

[43] Zekun Qi, Wenyao Zhang, Yufei Ding, Runpei Dong, Xinqiang Yu, Jingwen Li, Lingyun Xu, Baoyu Li, Xialin He, Guofan Fan, et al. Sofar: Language-grounded orientation bridges spatial reasoning and object manipulation. *arXiv preprint arXiv:2502.13143*, 2025. 2

[44] Yiran Qin, Zhelun Shi, Jiwen Yu, Xijun Wang, Enshen Zhou, Lijun Li, Zhenfei Yin, Xihui Liu, Lu Sheng, Jing Shao, et al. Worldsimbench: Towards video generation models as world simulators. *arXiv preprint arXiv:2410.18072*, 2024. 2

[45] Yiran Qin, Enshen Zhou, Qichang Liu, Zhenfei Yin, Lu Sheng, Ruimao Zhang, Yu Qiao, and Jing Shao. Mp5: A multi-modal open-ended embodied system in minecraft via active perception. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16307–16316. IEEE, 2024. 2

[46] Yiran Qin, Ao Sun, Yuze Hong, Benyou Wang, and Ruimao Zhang. Navigatediff: Visual predictors are zero-shot navigation assistants. *arXiv preprint arXiv:2502.13894*, 2025. 2

[47] Shreyas Sundara Raman, Vanya Cohen, Eric Rosen, Ifrah Idrees, David Paulius, and Stefanie Tellex. Planning with large language models via corrective re-prompting. *NeurIPS*, 2022. 2

[48] Vignesh Ramanathan, Anmol Kalia, Vladan Petrovic, Yi Wen, Baixue Zheng, Baishan Guo, Rui Wang, Aaron Marquez, Rama Kovvuri, Abhishek Kadian, et al. Paco: Parts and attributes of common objects. *CVPR*, 2023. 8

[49] Tianhe Ren, Shilong Liu, Ailing Zeng, Jing Lin, Kunchang Li, He Cao, Jiayu Chen, Xinyu Huang, Yukang Chen, Feng Yan, Zhaoyang Zeng, Hao Zhang, Feng Li, Jie Yang, Hongyang Li, Qing Jiang, and Lei Zhang. Grounded sam: Assembling open-world models for diverse visual tasks. *arXiv*, 2024. 5, 7

[50] Zhongwei Ren, Zhicheng Huang, Yunchao Wei, Yao Zhao, Dongmei Fu, Jiashi Feng, and Xiaojie Jin. Pixellm: Pixel reasoning with large multimodal model. *CVPR*, 2024. 7

[51] Kenneth Shaw, Ananye Agarwal, and Deepak Pathak. Leap hand: Low-cost, efficient, and anthropomorphic hand for robot learning. *arXiv*, 2023. 5, 7

[52] Zhelun Shi, Zhipin Wang, Hongxing Fan, Zaibin Zhang, Lijun Li, Yongting Zhang, Zhenfei Yin, Lu Sheng, Yu Qiao, and Jing Shao. Assessment of multimodal large language models in alignment with human values. *arXiv preprint arXiv:2403.17830*, 2024. 2

[53] Noah Shinn, Beck Labash, and Ashwin Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection. *arXiv*, 2023. 2

[54] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Cliport: What and where pathways for robotic manipulation. *CoRL*, 2021. 2, 5, 6

[55] Tom Silver, Soham Dan, Kavitha Srinivas, Joshua B Tenenbaum, Leslie Kaelbling, and Michael Katz. Generalized planning in pddl domains with pretrained large language models. *AAAI*, 2024. 2

[56] Marta Skreta, Naruki Yoshikawa, Sebastian Arellano-Rubach, Zhi Ji, Lasse Bjørn Kristensen, Kourosh Darvish, Alán Aspuru-Guzik, Florian Shkurti, and Animesh Garg. Errors are useful prompts: Instruction guided task programming with verifier-assisted iterative prompting. *arXiv*, 2023. 2

[57] Yingbo Tang, Shuaike Zhang, Xiaoshuai Hao, Pengwei Wang, Jianlong Wu, Zhongyuan Wang, and Shanghang Zhang. Affordgrasp: In-context affordance reasoning for open-vocabulary task-oriented grasping in clutter. *arXiv preprint arXiv:2503.00778*, 2025. 2

[58] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv*, 2023. 2

[59] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv*, 2023. 2

[60] Sai H Vemprala, Rogerio Bonatti, Arthur Bucker, and Ashish Kapoor. Chatgpt for robotics: Design principles and model abilities. *IEEE Access*, 2024. 3

[61] David Venuto, Sami Nur Islam, Martin Klissarov, Doina Precup, Sherry Yang, and Ankit Anand. Code as reward: Empowering reinforcement learning with vlms. *arXiv*, 2024. 3

[62] Homer Rich Walke, Kevin Black, Tony Z Zhao, Quan Vuong, Chongyi Zheng, Philippe Hansen-Estruch, Andre Wang He, Vivek Myers, Moo Jin Kim, Max Du, et al. Bridgedata v2: A dataset for robot learning at scale. *CoRL*, 2023. 5

[63] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v. *arXiv*, 2023. 3

[64] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv*, 2022. 2

[65] Runpeng Yu, Weihao Yu, and Xinchao Wang. Attention prompting on image for large vision-language models. *arXiv*, 2024. 3

[66] Zhouliang Yu, Jie Fu, Yao Mu, Chenguang Wang, Lin Shao, and Yaodong Yang. Multireact: Multimodal tools augmented reasoning-acting traces for embodied agent planning. *ICLR*, 2024. 2

[67] Wentao Yuan, Jiafei Duan, Valts Blukis, Wilbert Pumacay, Ranjay Krishna, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. Robopoint: A vision-language model for spatial affordance prediction for robotics. *arXiv*, 2024. 3

[68] Zhihao Yuan, Jinke Ren, Chun-Mei Feng, Hengshuang Zhao, Shuguang Cui, and Zhen Li. Visual programming for zero-shot open-vocabulary 3d visual grounding. *CVPR*, 2024. 3

[69] Jialiang Zhang, Haoran Liu, Danshi Li, XinQiang Yu, Haoran Geng, Yufei Ding, Jiayi Chen, and He Wang. Dexgraspnet 2.0: Learning generative dexterous grasping in large-scale synthetic cluttered scenes. *CoRL*, 2024. 5, 7

[70] Xinyu Zhang, Yuhan Liu, Haonan Chang, Liam Schramm, and Abdeslam Boularias. Autoregressive action sequence learning for robotic manipulation. *arXiv*, 2024. 5

[71] Yongting Zhang, Lu Chen, Guodong Zheng, Yifeng Gao, Rui Zheng, Jinlan Fu, Zhenfei Yin, Senjie Jin, Yu Qiao, Xuanjing Huang, et al. Spa-vl: A comprehensive safety preference alignment dataset for vision language model. *arXiv preprint arXiv:2406.12030*, 2024. 2

[72] Zaibin Zhang, Shiyu Tang, Yuanhang Zhang, Talas Fu, Yifan Wang, Yang Liu, Dong Wang, Jing Shao, Lijun Wang, and Huchuan Lu. Ad-h: Autonomous driving with hierarchical agents. *arXiv preprint arXiv:2406.03474*, 2024. 2

[73] Jinliang Zheng, Jianxiong Li, Sijie Cheng, Yinan Zheng, Jiaming Li, Jihao Liu, Yu Liu, Jingjing Liu, and Xianyuan Zhan. Instruction-guided visual masking. *arXiv*, 2024. 3

[74] Zhi Zheng, Qian Feng, Hang Li, Alois Knoll, and Jianxiang Feng. Evaluating uncertainty-based failure detection for closed-loop llm planners. *arXiv*, 2024. 2

[75] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ade20k dataset. *CVPR*, 2017. 8

[76] Enshen Zhou, Yiran Qin, Zhenfei Yin, Yuzhou Huang, Ruimao Zhang, Lu Sheng, Yu Qiao, and Jing Shao. Minedreamer: Learning to follow instructions via chain-of-imagination for simulated-world control. *arXiv preprint arXiv:2403.12037*, 2024. 2